

Modul Ajar Pemrograman Framework

Laravel 11

BAB 1. Pengenalan Laravel & Konsep MVC

Capaian Pembelajaran

1. Menjelaskan pengertian Laravel dan kelebihan penggunaan Framework Larave.
2. Memahami Gambaran arsitektur MVC (Model-View-Controller).
3. Menginstal Laravel dan menjalankan proyek dasar.
4. Memahami struktur direktori Laravel.

Materi

Apa itu Laravel

Laravel adalah framework PHP open-source yang digunakan untuk membangun aplikasi web secara cepat dan terstruktur. Laravel adalah sebuah framework PHP yang open source dan dirancang untuk kemudahan pengembangan aplikasi web. Framework Laravel mengikuti pola arsitektur Model-View-Controller (MVC) dan menyediakan berbagai fitur serta alat dan pustaka yang berguna untuk mempercepat proses pengembangan.

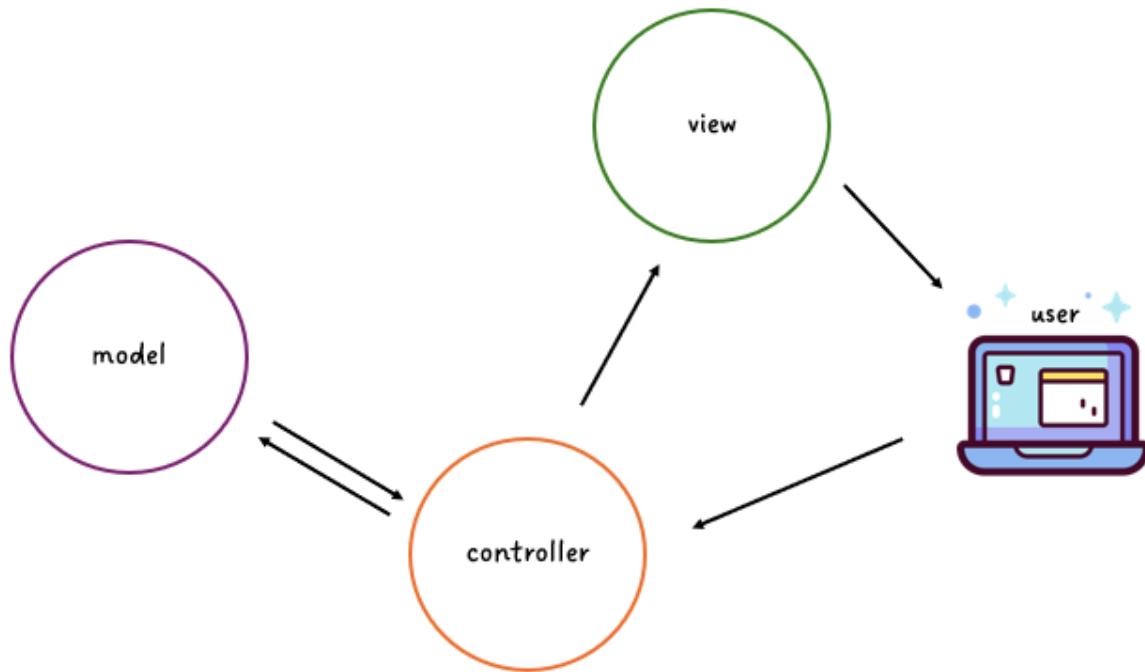
Framework: adalah kumpulan program berupa file pustaka (libraries) atau class-class yang mendukung dalam pengembangan aplikasi secara terstruktur dan independen terhadap aplikasi. Software Framework adalah sebuah desain yang bisa digunakan berulang-ulang (re-usable design) untuk sebuah sistem atau sub sistem piranti lunak.

Fitur Utama Laravel

1. **Routing** – Mengelola *URL* aplikasi dengan mudah.
2. **Eloquent ORM** – Berinteraksi dengan database menggunakan sintaks OOP, tanpa perlu menulis *query* SQL mentah.
3. **Blade Templating Engine** – Template HTML yang ringan dan powerful.
4. **Migration & Seeder** – Membuat struktur tabel dan dummy data menggunakan kode.
5. **Authentication & Authorization** – Login, register, dan hak akses dengan mudah dan middleware lainnya.
6. **RESTful API Support** – Mendukung pembuatan API backend.

Konsep Arsitektur MVC

Arsitektur Laravel didasarkan pada pola arsitektur Model-View-Controller (MVC) yang memisahkan logika aplikasi menjadi tiga komponen utama: **Model**, **View**, dan **Controller**



Komponen	Fungsi Utama	Letak di Laravel
Model	Mengelola data & logika bisnis, biasanya terhubung ke database	<code>app/Models</code>
View	Menampilkan antarmuka pengguna (HTML)	<code>resources/views</code>
Controller	Menangani permintaan pengguna & mengatur respon	<code>app/Http/Controllers</code>

Instalasi Laravel 11

Syarat Sistem:

- PHP $\geq$ 8.2
- Composer
- Web server (Laravel Valet / XAMPP / Laragon / Laravel Sail)

Tips : ada baiknya jika dalam pengerjaan proyek anda menggunakan distributed version control system GIT (GitHub.GitLab dll)

1. Install Laravel 11 via Composer

```

#jika ingin menginstall versi tertentu
composer create-project laravel/laravel:^11.0 nama-projek

#otomatis akan menginstall versi terbaru
  
```

```
composer create-project laravel/laravel nama-proyek
```

Sesuaikan nama proyek dengan kebutuhan anda. Jika menginstal tanpa menyebutkan versi maka composer akan menginstall versi terbaru dari laravel atau menginstall versi terbaru yang sesuai dengan PHP yang terinstal.

2. Masuk kedalam direktori proyek

```
cd nama-proyek
```

3. Jalankan Server Laravel

```
php artisan serve
```

Buka di browser <http://127.0.0.1:8000>, kalau muncul halaman Laravel, instalasi sukses

4. Konfigurasi Database (opsional)

Secara default Laravel menggunakan database sqlite. Jika anda ingin mengubah database sesuai dengan pilihan anda bisa di sesuaikan. Konfigurasi database bisa dilakukan di file `.env`

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=nama_database
DB_USERNAME=root
DB_PASSWORD=
```

5. Generate APP Key (jika belum otomatis)

```
php artisan key:generate
```

BAB 2. Routing dan Request Lifecycle

2.1 Capaian Pembelajaran

1. Menjelaskan konsep dasar Routing di Laravel.
2. Membuat rute sederhana hingga dinamis menggunakan Laravel.
3. Memahami Request Lifecycle (alur hidup permintaan) di Laravel.
4. Mengimplementasikan rute yang memanggil Controller dan View.

2.2 Materi

Routing di Laravel

Routing adalah mekanisme yang menentukan bagaimana Laravel merespons permintaan (request) ke URL tertentu.

Rute didefinisikan dalam file `routes/web.php` untuk rute berbasis web (HTTP) dan `routes/api.php` untuk rute berbasis API.

Routing Sederhana (Closure)

```
Route::get('/hello', function () {  
    return 'Halo, Laravel!';  
});
```

Routing dengan Parameter

```
Route::get('/employee/{id}', function ($id) {  
    return "Data pegawai dengan ID: " . $id;  
});
```

Routing dengan Parameter Opsional

```
Route::get('/employee/{nama?}', function ($nama = 'Tidak Diketahui') {  
    return "Nama Pegawai: " . $nama;  
});
```

Routing ke Controller

```
use App\Http\Controllers\EmployeeController;  
  
Route::get('/employee', [EmployeeController::class, 'index']);  
  
#default route
```

```
use App\Http\Controllers\EmployeeController;

Route::resource(employee, EmployeeController::class);
```

Routes yang Dihasilkan

Perintah di atas akan membuat **7 route** otomatis:

Method	URI	Action	Keterangan
GET	/employee	index	Tampilkan semua data
GET	/employee/create	create	Form tambah data
POST	/employee	store	Simpan data baru
GET	/employee/{ id }	show	Tampilkan detail
GET	/employee/{ id }/edit	edit	Form edit data
PUT/PATCH	/employee/{ id }	update	Update data
DELETE	/employee /{ id }	destroy	Hapus data

Pembatasan penggunaan route tertentu

Hanya Membuat Beberapa Route

```
Route::resource(employee, EmployeeController::class)->only([
    'index', 'show', 'store'
]);
```

Mengecualikan Beberapa Route

```
Route::resource(employee, EmployeeController::class)->except([
    'create', 'edit', 'destroy'
]);
```

Mengubah Nama Route

```
Route::resource(employee, EmployeeController::class)->names([
    'index' => 'posts.list',
    'show' => 'posts.view'
]);
```

API Resource Route

```
Route::apiResource(employee, EmployeeController::class);
```

Middleware Grup

```
Route::middleware(['auth'])->group(function () {
    Route::get('/employee', [EmployeeController::class, 'show']);
    Route::get('/settings', [SettingsController::class, 'index']);
});
```

Method HTTP yang Didukung Laravel

- GET → Mengambil data
- POST → Mengirim data baru
- PUT/PATCH → Memperbarui data
- DELETE → Menghapus data

Konsep Request Lifecycle (Alur Hidup Permintaan)

Request Lifecycle adalah proses yang dilalui sebuah request dari browser hingga Laravel memberikan response.

Tahapan Request Lifecycle di Laravel:

1. **User** mengakses URL di browser.
2. **public/index.php** menerima request dan mem-boot Laravel.
3. **HTTP Kernel** memproses request (menangani middleware global).
4. **Routing**: Laravel mencocokkan URL dengan definisi rute.
5. **Controller** atau Closure menangani logika permintaan.
6. **Model** digunakan jika membutuhkan data dari database.
7. **View** menampilkan hasil ke pengguna.
8. Laravel mengirim **response** kembali ke browser.

Middleware

Middleware merupakan mekanisme yang digunakan untuk melakukan **filtering** terhadap permintaan HTTP ke dalam aplikasi. Middleware adalah layer yang berada antara request dan response. Setiap request melewati middleware sebelum sampai ke controller.

Contoh sederhana digunakan untuk melakukan pengecekan terhadap user apakah sudah melakukan proses otentikasi. Jika belum maka middleware akan mengarahkan pada halaman *login*. Namun jika user sudah melakukan otentikasi, maka middleware akan mengizinkan permintaan ke dalam aplikasi.

Middleware tidak hanya digunakan untuk pengecekan otentikasi, namun bisa digunakan untuk semua jenis kondisi, misalnya menentukan halaman tertentu hanya bisa diakses jika user tersebut memiliki usia di atas 20 tahun, dan lain-lain. Semua middleware di Laravel disimpan di dalam folder `app/Http/Middleware`.

```
php artisan make:middleware CheckEmployeeAccess
```

Perintah di atas digunakan untuk membuat middleware baru dengan nama `CheckEmployeeAccess`. Jika perintah di atas berhasil maka akan membuat file baru `CheckEmployeeAccess.php` di dalam folder `app/Http/Middleware`.

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
```

```
class CheckEmployeeAccess
{
    public function handle(Request $request, Closure $next)
    {
        //Logika Middleware
        return $next($request);
    }
}
```

Pada **middleware** di atas ada method yang bernama **handle**. Method tersebut nantinya digunakan untuk menulis logika aplikasi dari middleware. Method handle tersebut memiliki 2 parameter, yaitu **Request \$request** dan **Closure \$next**.

Berikut contoh

```
public function handle(Request $request, Closure $next)
{
    if($request->age < 20 ) {
        return redirect('/home');
    }

    return $next($request);
}
```

Dari contoh diatas akan di cek apakah data umur yang dimasukan sudah 20 ke atas. Jika kondisi umur dibawah 20 tahun maka akan diarahkan ke *route* bernama **home**. Jika umur user 20 tahun keatas maka akan dilanjutkan ke Controller yang dituju.

Selajutnya adalah mendaftarkan middlleware bersifat **global**, yang artinya akan selalu dieksekusi saat ada permintaan **Http** ke aplikasi. Untuk itu middleware perlu di daftarkan dalam file **bootstrap/app.php**. cara untuk mendaftarkan adalah sebagai berikut.

```
use App\Http\Middleware\CheckEmployeeAccess;

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'CheckEmployeeAccess' => CheckEmployeeAccess::class
    ]);
})
```


Kode diatas adalah cara untuk membuat key `CheckEmployeeAccess` dan diarahkan ke Class middleware itu berada. Setelah mendaftarkan *middleware* maka sudah bisa digunakan pada Route, seperti pada contoh berikut.

```
Route::get('/employee, function () {  
    //  
})->middleware('CheckEmployeeAccess');
```

Dari contoh Route di atas artinya user yang bisa mengakses `/employee` adalah user yang memiliki usia di atas 20 tahun, karena telah ditambahkan middleware `CheckEmployeeAccess`

BAB 3. Controller

Capaian Pembelajaran

1. Menjelaskan fungsi dan peran Controller dalam arsitektur **MVC Laravel**.
2. Membuat Controller untuk mengatur logika aplikasi.
3. Memahami konsep **Blade Templating** di Laravel.
4. Membuat view dinamis menggunakan Blade.
5. Menghubungkan Controller dengan Blade View.

Materi

Controller di Laravel

Controller adalah bagian dari MVC yang bertugas menangani **logika aplikasi**. Controller menjadi penghubung antara **Model** (data) dan **View** (tampilan). Semua Controller Laravel disimpan di direktori `app/Http/Controllers`

Membuat Controller

Gunakan perintah artisan dengan perintah :

```
# normal controller  
php artisan make:controller PostController  
  
# resource controller (CRUD)  
php artisan make:controller PhotoController --resource  
  
# Membuat controller dengan model sekaligus
```

```
php artisan make:controller EmployeeController --resource --model=Employee
```

```
# Controller API resource
```

```
php artisan make:controller EmployeeController --api
```

Parintah di atas digunakan untuk menghasilkan *Controller* baru dengan nama **EmployeeController.php** yang berada pada folder **app/Http/Controllers**.

Struktur Dasar Kontroller

Jika perintah membuat Kontroller berhasil dijalankan, maka Laravel akan mengenerate Controller secara otomatis. Berikut adalah hasil luaran Controller yang telah dibuat.

```
namespace App\Http\Controllers;

use App\Models\Employee;
use Illuminate\Http\Request;

class EmployeeController extends Controller
{
    // Method-method akan ditambahkan di sini
}
```

```
namespace App\Http\Controllers;

use App\Models\Employee;
use Illuminate\Http\Request;

class EmployeeController extends Controller
{
    public function index() {}           // GET /employees
    public function create() {}          // GET /employees/create
    public function store() {}           // POST /employees
    public function show() {}            // GET /employees/{id}
    public function edit() {}            // GET /employees/{id}/edit
    public function update() {}          // PUT/PATCH /employees/{id}
    public function destroy() {}         // DELETE /employees/{id}
}
```

Hasil di atas adalah hasil dari generate Controller. Jika menggunakan perintah resource laravel langsung menyediakan fungsi bawaan untuk proses CRUD. Hal ini tentu akan sangat membantu pengguna jika ingin membuat fungsi CRUD dari controller yang dibuat.

Request

Laravel memiliki Class `Illuminate\Http\Request` yang digunakan untuk menerima Http Request yang dikirim, seperti `input`, `cookies`, `files`, dan lainnya. Berikut contoh sederhana penggunaan request untuk mendapatkan data dari form yang dikirim oleh user.

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class EmployeeController extends Controller
{
    public function store(Request $request)
    {
        $name = $request->input('name');
    }
}
```

Pada contoh diatas yang pertama di import adalah class `Http Request`. Kemudian di dalam method `store` terdapat parameter `Request $request`. Tujuannya adalah agar bisa mengirim request ke dalam method `store`. Di dalamnya terdapat *variable* bernama `$name` yang berisi request `input` dengan nama `name` yang dikirim oleh *user*.

List Request

Request	Deskripsi
<code>\$request->all()</code>	Mengambil semua <i>request</i> yang dikirim oleh <i>user</i>
<code>\$request->input('name')</code>	Mengambil request <code>input</code> bernama <code>name</code> yang dikirim oleh <i>user</i>
<code>\$request->query('keywords')</code>	Mengambil request <code>query</code> bernama <code>keywords</code> yang ada di <i>URL</i>
<code>\$request->only(['username', 'password'])</code>	Mengambil hanya data <code>username</code> dan <code>password</code> yang dikirim oleh <i>user</i>
<code>\$request->except(['credit_card'])</code>	Mengambil kecuali data <code>credit_card</code> yang dikirim <i>user</i>

<code>\$request->file('photo')</code>	Mengambil request file bernama photo yang dikirim <i>user</i>
<code>\$request->cookie('name')</code>	Mengambil request cookies bernama name yang dikirim dari <i>browser</i>
<code>\$request->date('birthday')</code>	Mengambil request date bernama birthday yang dikirim <i>user</i>

Responses

Semua **route** dan **controller** harus mengembalikan sebuah **response** untuk ditampilkan ke pengguna pada browser. Laravel memiliki beberapa cara untuk mengembalikan response. Response paling dasar dan umum digunakan adalah mengembalikan string dari *route* atau *controller*

Views

Views di Laravel adalah komponen dalam arsitektur MVC yang bertanggung jawab untuk menampilkan antarmuka pengguna atau konten di browser. **Views** berisi kode HTML yang digabungkan dengan bahasa templating **Blade**, yang memungkinkan pengembang untuk membuat tampilan yang dinamis dan terpisah dari logika bisnis, serta memudahkan pengelolaan komponen berulang seperti header dan footer. Semua file **views** akan ditempatkan pada direktori **resources/views**.

Contoh sederhana penggunaan views untuk menampilkan data.

```
Route::get('/', function () {
    return view('home', ['name' => 'Muarifin']);
});
```

Kode di atas adalah membuat route dengan path **/** dan di dalamnya melakukan return 2 parameter ke dalam **view**. Parameter yang pertama adalah **home** yang mana ini adalah nama file viewnya, sedangkan **name => Muarifin** adalah data yang dikirim ke dalam view.

Selanjutnya adalah menyiapkan file *viewnya* yaitu **resources/views/home.blade.php** untuk menampilkan datanya, misalkan seperti berikut :

```
<html>
  <body>
    <h1>Hello, {{ $name }}</h1>
  </body>
</html>
```

Jika yang dilakukan betul ketika dibuka pada browser `127.0.0.1:8000` maka akan muncul text "Muarifin" pada browser.

Validation

Validation merupakan hal yang sangat penting saat membuat aplikasi. **Validation** merupakan metode yang digunakan laravel yang digunakan untuk memvalidasi permintaan *Http* yang masuk dengan aturan validasi.

Contoh sederhana pembuatan validasi :

```
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required|unique:posts|max:255',
        'jabatan' => 'required',
    ]);

    // The Employee is valid...
}
```

Kode diatas adalah untuk melakukan validasi dengan menggunakan method validate dan didalamnya kita bisa memberikan validasi yang diinginkan.

BAB 4. Bekerja dengan Database

Materi

Migration

Migration di Laravel adalah fitur untuk mengelola struktur database dengan kode (bukan manual di phpMyAdmin/SQL). Bisa membuat tabel, mengubah kolom, menambahkan relasi, bahkan rollback ke versi sebelumnya. *Migration* adalah merupakan *Version Control* database, sehingga dengan menggunakan migration developer tidak perlu menggunakan aplikasi tambahan misal *PhpMyAdmin* dalam membuat tabel, field dan komponen database lainnya.

Keuntungan Migration:

- Version control database schema
- Kolaborasi tim lebih mudah
- Deployment konsisten across environments
- Rollback changes jika ada error

Membuat Migration

```
# Membuat migration untuk tabel employees
```

```
php artisan make:migration create_employees_table
```

Membuat migration dengan spesifik tabel

```
php artisan make:migration create_employees_table --create=employees
```

Struktur Dasar Migration

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    # Run the migrations.
    public function up(): void
    {
        Schema::create('employees', function (Blueprint $table) {
            $table->id(); // BIGINT unsigned auto_increment primary key
            $table->timestamps(); // created_at & updated_at
        });
    }

    # Reverse the migrations.
    public function down(): void
    {
        Schema::dropIfExists('employees');
    }
};
```

Jika diperhatikan kode di atas terdapat 2 method bawaan yaitu **function up** dan **function down**. Method **function up** digunakan untuk menambahkan *field-field* yang akan dibuat dalam *tabel*, sedangkan method **function down** akan menghapus tabel

Untuk menjalankan **migration** menggunakan perintah **php artisan migrate**. Ketika menjalankan perintah tersebut maka *file-file migration* yang ada akan menjalankan *method function up*, sehingga table-table beserta fieldnya akan digenerate kedalam database. Jika ingin mengembalikan versi dimana sebelum menjalankan migrasi sebelumnya satu tingkat bisa menggunakan perintah berikut **php artisan migrate:rollback**

Query Builder

Query Builder merupakan fitur dalam laravel yang digunakan untuk menjalankan sebuah *query* kedalam *database* dengan lebih mudah dan sederhana. *Query Builder* biasanya digunakan untuk menjalankan query yang sangat kompleks dan query yang sudah susah ditangani oleh **Eloquent**. *Query Builder* menggunakan sistem *parameter binding* yang ada didapam PDO, sehingga akan lebih aman jika terkena serangan *SQL injection*.

Contoh implementasi query builder :

```
use Illuminate\Support\Facades\DB;

$users = DB::table('users')->get();
```

Dari contoh di atas tampak bahwa jika menggunakan *query builder* terlebih dahulu harus melakukan *import facades DB* terlebih dahulu melalui `use Illuminate\Support\Facades\DB`. Selanjutnya baru bisa mengambil record dengan method bawaan *query builder* pada contoh di atas menggunakan method `get()`

Contoh menampilkan spesifik tabel

```
$users = DB::table('employee')
    ->select('name', 'email')
    ->get();
```

Dari contoh di atas mengambil *record* dengan menggunakan method `select` dan berisi field tertentu yang ingin di tampilkan yaitu field `name` dan `email`.

Untuk mempelajari berbagai macam penggunaan Query Builder bisa Anda pelajari pada Dokumentasinya pada tautan berikut <https://laravel.com/docs/11.x/queries>

Eloquent

Eloquent adalah fitur Object-Relational Mapper (ORM) bawaan dari framework PHP **Laravel** yang memungkinkan pengembang untuk berinteraksi dengan database menggunakan *objek* PHP. Eloquent menyederhanakan dan mempercepat operasi dasar *database* seperti membuat, membaca, memperbarui, dan menghapus data (CRUD), serta menyediakan sintaks yang lebih *ekspresif* dan *mudah dibaca*. Dalam menggunakan *Eloquent* selalu membutuhkan *Model* yang terkait dengan tabel, sehingga proses seperti `insert`, `update`, `delete` dll menjadi sangat mudah.

Cara membuat Model :

```
php artisan make:model Employee
```

Untuk membuat model pada laravel bisa menggunakan perintah artisan yaitu `php:make`. Seperti pada kode di atas akan membuat model yang baru bernama `Employee.php` dan berada di dalam direktori `app/Models`.

Berikut adalah contoh penggunaan Eloquent

Model sudah berhasil dibuat, maka *Eloquent* bisa digunakan untuk berbagai kasus, misalnya ingin menampilkan semua record dari *tabel* tertentu. Berikut adalah beberapa contoh penggunaan *Eloquent*. Untuk belajar detail Eloquent bisa masuk pada tautan berikut : <https://laravel.com/docs/11.x/eloquent>

```
Employee::all(10);
```

Kode di atas digunakan untuk mencari semua *record* yang ada di *tabel* `employee` dan menggunakan method `all`

```
Employee::paginate(10);
```

Kode di atas digunakan untuk menampilkan *record* yang berisi sejumlah data disetiap halamannya dengan menggunakan method `paginate`.

```
Employee::find(1);
```

Kode di atas digunakan untuk menampilkan *record* berdasarkan primary key. Contoh di atas untuk menampilkan data dari *tabel* `employee` yang memiliki primary key `1`

```
Employee::create([
    'name'      => $request->name,
    'address'   => $request->address
]);
```

```
$emp = Employee::find(1);
```

```
$emp->update([
    'name'      => $request->name,
    'address'   => $request->address
]);
```

```
$emp = Employee::find(1);
```

```
$emp->delete();
```

Kode di atas digunakan untuk proses manipulasi data ke dalam *tabel* `employee`. *Eloquent* memudahkan anda untuk melakukan manipulasi data (`insert`, `update`, `delete`) ke dalam database menggunakan `model`.

BAB 5. Relasi Database (Eloquent ORM Part 2)

Materi

BAB 6. Blade dan Templating

Materi

BAB 7. Seeder

Materi

BAB 8. Autentikasi dan Middleware

Materi

BAB 9. REST API Dasar di Laravel

Materi

BAB 10. Deployment

Materi

Langkah Kerja

Latihan

Referensi

Laravel

1. [PetaniKode – Tutorial Laravel 11](#)
2. [Santri Koding – Tutorial Laravel 10 Pemula](#)
3. [Malas Ngoding – Tutorial Laravel 5.7](#)
4. [Programmer Zaman Now – Tutorial Laravel 10](#)
5. [Parsinta – Tutorial Laravel 11](#)
6. [Net Ninja - Tutorial Laravel 11](#)
7. [Web Programmer UNPAS – Tutorial Laravel 11](#)

Git

1. [Malas Ngoding- Tutorial Git](#)
2. [Programer UNPAS – Tutorial Git](#)
3. [Learningbranching - Simulasi Git Online](#)

Persiapan Kuliah

Dalam penggunaan pembelajaran ini akan menggunakan studi kasus struktur database berikut :

NO	Tabel	Deskripsi
1	Karyawan (employees)	Menyimpan data pribadi karyawan.
2	Departemen (departments)	Setiap karyawan berhubungan dengan satu departemen.
3	Jabatan (positions)	Menyimpan informasi tentang jabatan yang dipegang oleh karyawan.
4	Absensi (attendance)	Menyimpan data kehadiran harian karyawan.
5	Gaji (salaries)	Menyimpan riwayat gaji dan tunjangan karyawan.

Desain Tabel

Tabel **employees**

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
nama_lengkap	VARCHAR(100)	Nama lengkap karyawan
email	VARCHAR(100)	Email karyawan (unik)
nomor_telepon	VARCHAR(15)	Nomor telepon
tanggal_lahir	DATE	Tanggal lahir
alamat	TEXT	Alamat karyawan
tanggal_masuk	DATE	Tanggal mulai bekerja
departement_id	INT	Foreign Key ke departments
jabatan_id	INT	Foreign Key ke positions
status	ENUM('aktif', 'nonaktif')	Status karyawan

Tabel **departments**

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
nama_departemen	VARCHAR(100)	Nama departemen

Tabel **positions**

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
nama_jabatan	VARCHAR(100)	Nama jabatan
gaji_pokok	DECIMAL(10,2)	Gaji pokok untuk jabatan ini

Tabel **attendance**

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
karyawan_id	INT	Foreign Key ke employees
tanggal	DATE	Tanggal absensi
waktu_masuk	TIME	Waktu masuk kerja
waktu_keluar	TIME	Waktu pulang kerja
status_absensi	ENUM('hadir', 'izin', 'sakit', 'alpha')	Status kehadiran

Tabel **salaries**

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
karyawan_id	INT	Foreign Key ke employees
bulan	VARCHAR(10)	Bulan dan tahun pembayaran
gaji_pokok	DECIMAL(10,2)	Gaji pokok
tunjangan	DECIMAL(10,2)	Total tunjangan
potongan	DECIMAL(10,2)	Potongan (misal: BPJS, pajak)
total_gaji	DECIMAL(10,2)	Total gaji yang diterima

Hubungan Tabel

- **employees** berelasi dengan **departments** melalui *departemen_id* (many-to-one).
- **employees** berelasi dengan **positions** melalui *jabatan_id* (many-to-one).
- **attendance** berelasi dengan **employees** melalui *karyawan_id* (many-to-one).
- **salaries** berelasi dengan **employees** melalui *karyawan_id* (many-to-one).

Tips : ada baiknya jika dalam pengerjaan proyek anda menggunakan distributed version control system GIT (GitHub, GitLab dll)

Latihan Studi Kasus

Setting Environment dan Instalasi

- Setting Environment

Pastikan bahwa Environment kebutuhan Laravel 11 sudah siap.

6. Tentukan Direktori Proyek

Tentukan dimana proyek Laravel Anda, misalkan pada drive `D:/laragon/www`

7. Install via Composer

Buka Command Prompt dan arahkan pada direktori tersebut. Pada proyek ini menggunakan Laravel ver 11

```
C:\Windows\system32\cmd.exe

D:\laragon>cd ..

D:\>cd laragon\www

D:\laragon\www>composer create-project laravel/laravel:^11.0 app-pegawai_
```

8. Menjalankan server Laravel

Mau ke direktori proyek anda dan jalankan perintah berikut:

```
C:\Windows\system32\cmd.exe - php artisan serve

D:\laragon\www>
D:\laragon\www>cd app-pegawai

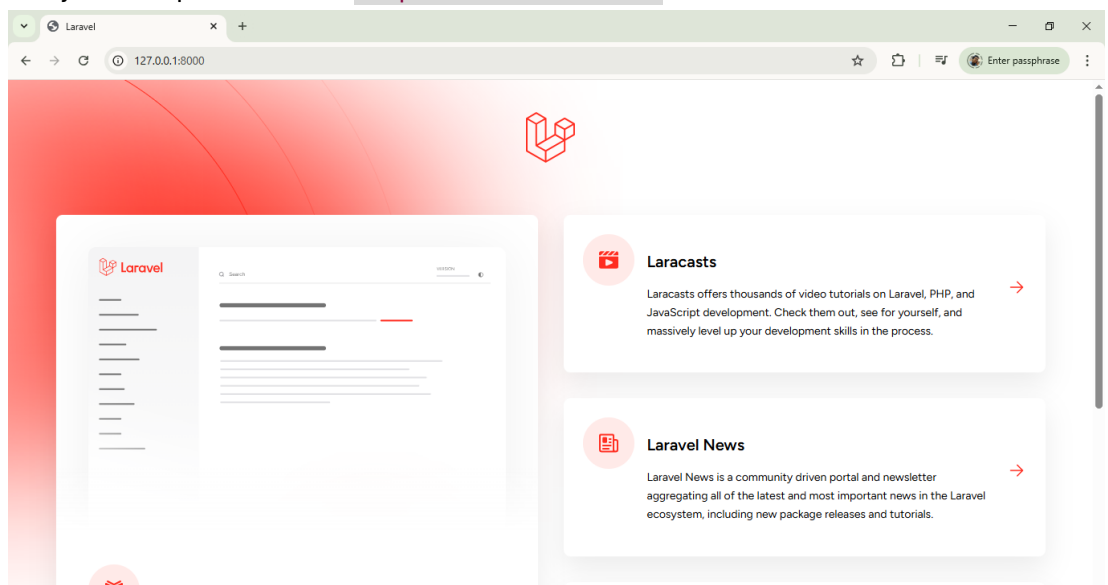
D:\laragon\www\app-pegawai>php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server
```

9. Ujicoba Menjalankan Program

Menjalankan pada browser `http://127.0.0.1:8000`

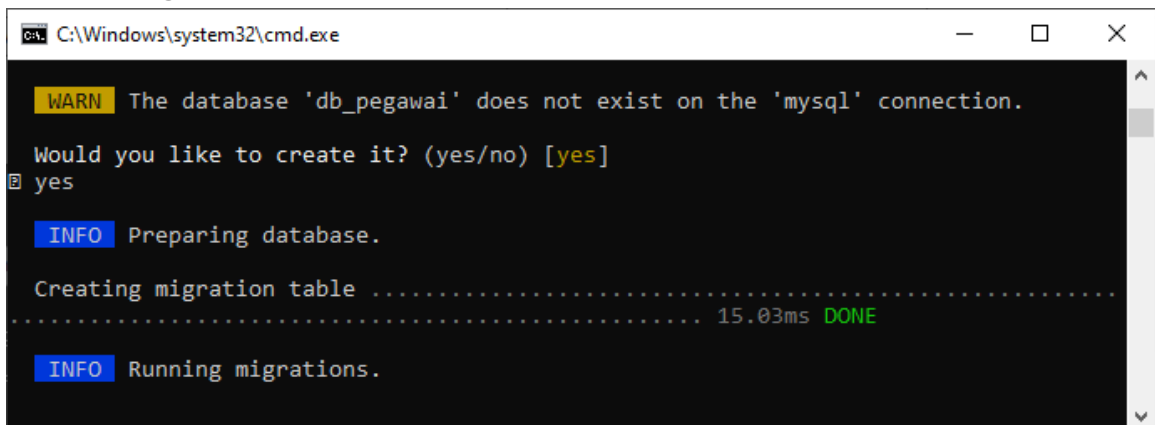


10. Konfigurasi Database MySQL

Buka direktori proyek Anda dengan menggunakan IDE favorit anda, misalkan Visual Studio Code. Buka file .env dan lakukan konfigurasi database mysql. Sesuaikan username dan password database Anda.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_pegawai
DB_USERNAME=root
DB_PASSWORD=
```

11. Jalankan Migrasi



```
C:\Windows\system32\cmd.exe

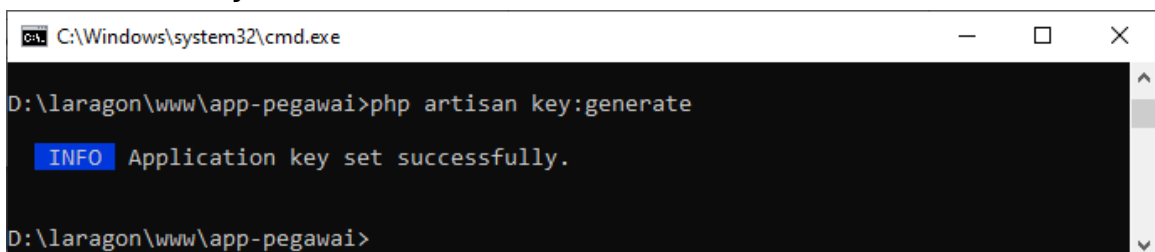
WARN The database 'db_pegawai' does not exist on the 'mysql' connection.
Would you like to create it? (yes/no) [yes]
yes

INFO Preparing database.

Creating migration table ..... 15.03ms DONE

INFO Running migrations.
```

12. Membuat APP Key



```
C:\Windows\system32\cmd.exe

D:\laragon\www\app-pegawai>php artisan key:generate

INFO Application key set successfully.

D:\laragon\www\app-pegawai>
```

13. Jalankan server laravel dan akses kembali pada browser. Jika muncul dan tidak ada eror berarti anda berhasil

Tugas ke 1

1. Buat akun pada *github*.
2. Buat Proyek laravel 11 dengan nama `app-pegawai`
3. Setelah berhasil menjalankan pertama kali unggah (`push`) proyek anda pada *repository github*
4. Lakukan konfigurasi Proyek Anda ke database MySQL/Postgre
5. Unggah(push) kembali pada Repositori github dan berikan pesan `commit` "Konfigurasi Awal Proyek"

Migration dan Model

Bekerja pada Repository Git

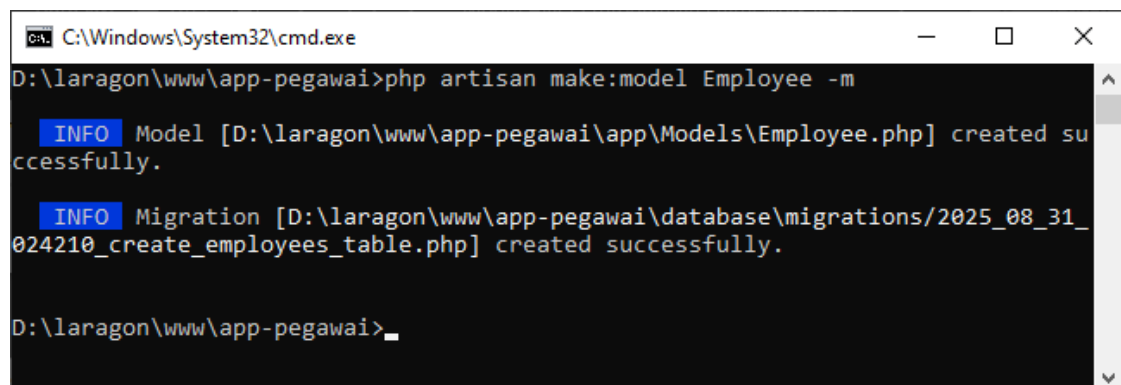
1. Buat *branch* `dev` pada repository lokal anda drnga perintah `git branch dev`
2. Masuk ke *branch* `dev` dengan perintah `git checkout dev`. Perintah tersebut digunakan untuk masuk ke *branch*(*cabang*) developepment pada repository. Tujuan pimisahan cabang ini adalah cara sederhana untuk memisahkan antara proyek yang siap release dan masih pengembangan. Pada *branch* `main` berisi `commit` proyek anda yang sudah siap direlease dan pada *branch* `dev` berisi proyek anda yang masih tahap pengembangan.
3. Sebelum masuk ke pengembangan proyek pada *branch* `dev`. Setelah *checkout* buat `commit` baru pembuatan branch baru dengan perintah

```
git add .
git commit -m "buat branch baru dev"
git push -u origin dev
```

Parameter `-u` adalah **upstream** untuk pertama kali melakukan **push** ke *remote repository*. Selanjutnya jika ingin **push** ke *remote repository* cukup menggunakan `git push origin dev`.

Selanjutnya kita bekerja pada proyek Laravel Anda

4. Membuat *model* dan *migration* tabel `Employee`. Dalam membuat *model* dan *migration* bisa dilakukan secara terpisah atau langsung sekaligus dalam perintah `artisan`. Jika membuat sekaligus dengan perintah `artisan` maka penamaan *migration* dibuat otomatis. Pada percobaan ini akan menggunakan perintah membuat model sekaligus migration. Masuk ke *command prompt* pada proyek dan ketik perintah berikut `php artisan make:model Employee -m`. parameter `-m` adalah perintah artisan membuat migration sekaligus ketika membuat model.



```
C:\Windows\System32\cmd.exe
D:\laragon\www\app-pegawai>php artisan make:model Employee -m

[INFO] Model [D:\laragon\www\app-pegawai\app\Models\Employee.php] created successfully.

[INFO] Migration [D:\laragon\www\app-pegawai\database\Migrations\2025_08_31_024210_create_employees_table.php] created successfully.

D:\laragon\www\app-pegawai>
```

Jika berhasil maka artisan akan membuat file Model dan Migration dan bisa dilihat pada direktori berikut :

```
- app/Models/Employee.php
- database/migrations/2025_08_31_024210_create_employees_table.php
```

5. Masuk ke file *migration* dan menuju ke `function up` untuk mengisi properti tabel Employee sesuai dengan **Tabel Desain(employee)**, untuk sementara hilangkan **Foreign Key** karena tabel masternya belum ada sehingga menjadi seperti berikut :

```
Schema::create('employees', function (Blueprint $table) {
    $table->id();
    $table->string('nama_lengkap', 100);
    $table->string('email', 100);
    $table->string('nomor_telepon', 15);
    $table->date('tanggal_lahir');
    $table->text('alamat');
    $table->date('tanggal_masuk');
    $table->enum('status', ['aktif', 'nonaktif'])->default ('aktif');
    $table->timestamps();
});
```

6. Konfigurasi **Mass Assignment** pada Model `Employee`. Tujuannya agar tabel dapat dilakukan menyimpan data. Semua field pada tabel selain `id` dan `timestamps` harus dimasukkan.

```
class Employee extends Model
{
    protected $fillable = [
        'nama_lengkap',
        'email',
        'nomor_telepon',
        'tanggal_lahir',
        'alamat',
        'tanggal_masuk',
        'status',
    ];
}
```

7. Jalankan perintah artisan migration `php artisan migrate` jika berhasil maka tabel `employee` sudah otomatis dibuat pada database.

```
C:\Windows\System32\cmd.exe

D:\laragon\www\app-pegawai>php artisan migrate

  INFO  Running migrations.

 2025_08_31_024210_create_employees_table ..... 32.42ms DONE

D:\laragon\www\app-pegawai>
```

Silahkan cek pada database MySQL anda jika berhasil tabel employee sudah berhasil dibuat.

Tugas ke 2

1. Kerjakan Percobaan di atas
2. Lakukan commit pada lokal repository anda pada *branch* `dev` dengan perintah `git add .` selanjutnya `git commit -m "penambahan migration dan model employee"`
3. Push ke remote repository anda. Pastikan sudah di *branch* `dev` lalu jalankan perintah `push origin dev`

CRUD

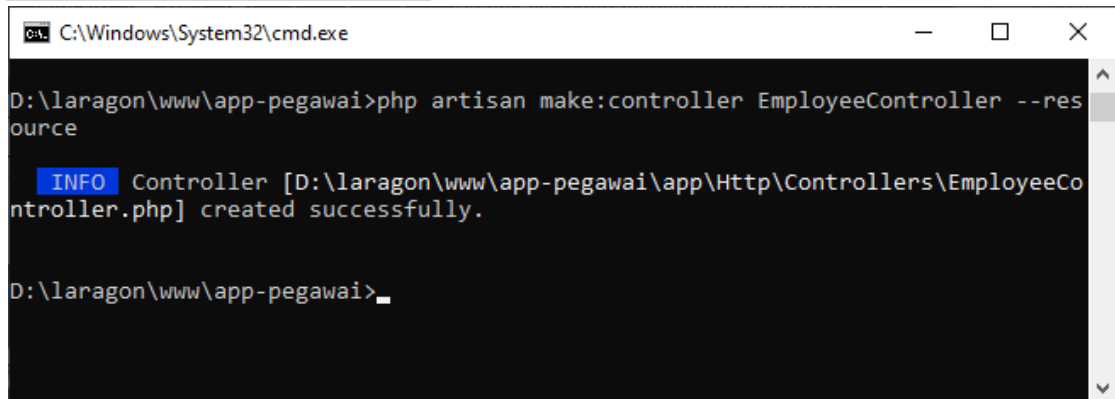
Menyiapkan Tampilan

Blade adalah template engine bawaan Laravel. **Blade** digunakan untuk memisahkan logika aplikasi (Controller & Model) dengan tampilan (View). File Blade disimpan di folder: `resources/views/`. Ekstensi file: `.blade.php`. Pada percobaan ini kita akan menyiapkan dulu template bade sederhana sebagai wadah tampilan untuk proses CRUD tabel `employee`. Dengan pemahaman cara kerja template engine balde maka akan memudahkan pengelolaan tampilan laravel kedepanya.

Menampilkan Data

Controller ini yang nanti akan menerima permintaan `http` dari `user` yang di arahkan melalui `route`, memproses logika aplikasi dan memberikan respon.

1. **Buat Controller** `Employee` dengan perintah `php artisan make:controller EmployeeController --resource`.



```
C:\Windows\System32\cmd.exe

D:\laragon\www\app-pegawai>php artisan make:controller EmployeeController --resource

INFO Controller [D:\laragon\www\app-pegawai\app\Http\Controllers\EmployeeController.php] created successfully.

D:\laragon\www\app-pegawai>
```

Jika berhasil maka artisan akan membuat file Controller dan dapat dilihat pada `app/Http/Controllers/EmployeeController.php`. Penggunaan `--resource` adalah untuk membuat generate otomatis method yang diperlukan untuk proses CRUD.

2. Buka Controller `EmployeeController.php` dan buat kode untuk mengambil data dari database pada method `index`



```
<?php
namespace App\Http\Controllers;

use App\Models\Employee;
use Illuminate\Http\Request;
class EmployeeController extends Controller
{
    public function index()
    {
        $employees = Employee::latest()->paginate(5);

        return view('employees.index', compact('employees'));
    }
}
```

Yang pertama harus dilakukan adalah melakukan import Model Employee dengan key word `use` yaitu `App\Models\Employee`. Selanjutnya dengan menggunakan Eloquent memanggil Objek `Employee` untuk mengambil data dari tabel employees. Saat mengirim data ke `views`

menggunakan fungsi `compact` yang akan membuat *array asosiatif* dari variabel. Selain menggunakan `compact` ada banyak cara lain yang bisa digunakan untuk mengirim data ke view yaitu `array`, `with`, `make` dll

3. Membuat file **view** pada direktori `views` tambahkan direktori `employeee` dan membuat file `index.blade.php` sebagai file **view** untuk menampilkan data employee.

```
<!DOCTYPE html>
<html lang="en">
<head>

</head>
<body>
    <div class="container mt-5">
        <h1 class="mb-4">Daftar Pegawai</h1>
        <table border="1" cellpadding="5" cellspacing="0">
            <thead>
                <tr>
                    <th>Nama Lengkap</th>
                    <th>Email</th>
                    <th>Nomor Telepon</th>
                    <th>Tanggal Lahir</th>
                    <th>Alamat</th>
                    <th>Tanggal Masuk</th>
                    <th>Status</th>
                    <th>Aksi</th>
                </tr>
            </thead>
```

```

<tbody>
  @foreach($employees as $employee)
    <tr>
      <td>{{ $employee->nama_lengkap }}</td>
      <td>{{ $employee->email }}</td>
      <td>{{ $employee->nomor_telepon }}</td>
      <td>{{ $employee->tanggal_lahir }}</td>
      <td>{{ $employee->alamat }}</td>
      <td>{{ $employee->tanggal_masuk }}</td>
      <td>{{ $employee->status }}</td>
      <td>
        <a href="{{ route('employees.show', $employee->id) }}">Detail</a> |
        <a href="{{ route('employees.edit', $employee->id) }}">Edit</a> |
        <form action="{{ route('employees.destroy', $employee->id) }}" method="POST" style="display:inline;">
          @csrf
          @method('DELETE')
          <button type="submit" onclick="return confirm('Yakin ingin menghapus?')>Delete</button>
        </form>
      </td>
    </tr>
  @endforeach
</tbody>
</table>
</div>
</body>
</html>

```

4. Menambahkan route pada `routes/web.php` dengan menggunakan **resource**, sehingga tidak perlu membuat route satu persatu. Import Controllers dengan ketik code berikut `use App\Http\Controllers\EmployeeController`.

```
Route::resource('employees',EmployeeController::class);
```

5. Jalankan pada browser `http://127.0.0.1:8000/employees`



Daftar Pegawai

Insert Data

1. Untuk memasukan data ke dalam **tabel** dalam **database** yang harus dilakukan adalah dengan menambahkan kode pada method **create** (menampilkan form insert) dan **store** (menambahkan kode untuk **insert** data ke tabel).
2. Buka file **EmployeeController.php** dan tambahkan kode berikut pada method **create**

```
public function create()
{
    return view('employees.create');
}
```

3. Pada file **EmployeeController.php** dan tambahkan kode berikut pada method **store**

```
public function store(Request $request)
{
    $request->validate([
        'nama_lengkap' => 'required|string|max:255',
        'email'         => 'required|email|max:255',
        'nomor_telepon'  => 'required|string|max:20',
        'tanggal_lahir'  => 'required|date',
        'alamat'         => 'required|string|max:255',
        'tanggal_masuk'  => 'required|date',
        'status'         => 'required|string|max:50',
    ]);
    Employee::create($request->all());
    return redirect()->route('employees.index');
}
```

4. Buat file `create.blade.php` dalam direktori `employees` untuk membuat **Form** input data. Tambahkan kode berikut.

```
<!DOCTYPE html>
<html>
<head>
    <title>Form Input Pegawai</title>
</head>
<body>
    <h1 class="mb-4">Form Pegawai</h1>
    <form action="{{ route('employees.store') }}" method="POST">
        @csrf
        <table>
            <tr>
                <td><label for="nama_lengkap">Nama Lengkap:</label></td>
                <td><input type="text" id="nama_lengkap" name="nama_lengkap"></td>
            </tr>
            <tr>
                <td><label for="email">Email:</label></td>
                <td><input type="email" id="email" name="email"></td>
            </tr>
            <tr>
                <td><label for="nomor_telepon">Nomor Telepon:</label></td>
                <td><input type="text" id="nomor_telepon" name="nomor_telepon"></td>
            </tr>
            <tr>
                <td><label for="tanggal_lahir">Tanggal Lahir:</label></td>
```

```

        <td><input type="date" id="tanggal_lahir" name="tanggal_lahir"></td>
    </tr>
    <tr>
        <td><label for="alamat">Alamat:</label></td>
        <td><textarea id="alamat" name="alamat"></textarea></td>
    </tr>
    <tr>
        <td><label for="tanggal_masuk">Tanggal Masuk:</label></td>
        <td><input type="date" id="tanggal_masuk" name="tanggal_masuk"></td>
    </tr>
    <tr>
        <td><label for="status">Status:</label></td>
        <td>
            <select id="status" name="status">
                <option value="aktif">Aktif</option>
                <option value="nonaktif">Nonaktif</option>
            </select>
        </td>
    </tr>
    <tr>
        <td colspan="2" style="text-align:right;">
            <button type="submit">Simpan</button>
        </td>
    </tr>
</table>
</form>

```



```
</body>  
</html>
```

Kode di atas menggunakan html biasa tanpa CSS hanya menggunakan tabel untuk merapikan form agar memudahkan pembelajaran.

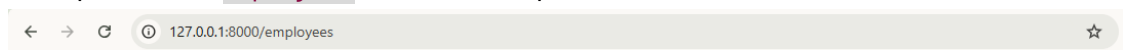
5. Ujicoba Input pada **Form** dengan mengakses `127.0.0.1:8000/employees/create`



Form Pegawai

Nama Lengkap:	Muarifin
Email:	muarifin@pens.ac.id
Nomor Telepon:	085815667668
Tanggal Lahir:	03/12/1988
Alamat:	Taman Sukodono Regency Cluster 1
Tanggal Masuk:	21/09/2025
Status:	Aktif
Simpan	

6. **Klik** simpan jika berhasil akan di-redirect ke halaman utama jika tidak diredirect silahkan buka pada **route** `employees` untuk menampilkan data.



Daftar Pegawai

Nama Lengkap	Email	Nomor Telepon	Tanggal Lahir	Alamat	Tanggal Masuk	Status	Aksi
Muarifin	muarifin@pens.ac.id	085815667668	1988-12-03	Taman Sukodono Regency Cluster 1	2025-09-21	aktif	Detail Edit Delete

Menampilkan detail Data

1. Menampilkan **detail** data menggunakan method `show`
2. Buka file `EmployeeController.php` dan tambahkan kode berikut pada method `show`.

```
public function show(string $id)
{
    $employee = Employee::find($id);
    return view('employees.show', compact('employee'));
}
```

3. Buat file `show.blade.php` dalam direktori `employees` untuk membuat **tampilan data secara detail**. Tambahkan Kode berikut :

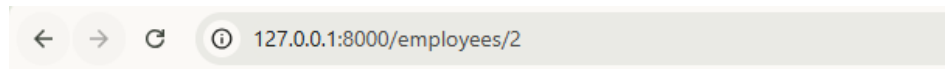
```
<!DOCTYPE html>
<html>
<head>
    <title>Detail Pegawai</title>
</head>
<body>
    <h1>Detail Pegawai</h1>
    <table border="1" cellpadding="8" cellspacing="0">
        <tr>
            <th>Nama Lengkap</th>
            <td>{{ $employee->nama_lengkap }}</td>
        </tr>
        <tr>
            <th>Email</th>
            <td>{{ $employee->email }}</td>
        </tr>
        <tr>
            <th>Nomor Telepon</th>
            <td>{{ $employee->nomor_telepon }}</td>
        </tr>
        <tr>
            <th>Tanggal Lahir</th>
            <td>{{ $employee->tanggal_lahir }}</td>
        </tr>
        <tr>
            <th>Alamat</th>
```

```

        <td>{{ $employee->alamat }}</td>
    </tr>
    <tr>
        <th>Tanggal Masuk</th>
        <td>{{ $employee->tanggal_masuk }}</td>
    </tr>
    <tr>
        <th>Status</th>
        <td>{{ $employee->status }}</td>
    </tr>
</table>
</body>
</html>

```

4. **Ujicoba** dengan masuk ke halaman utama menampilkan data pada route <http://127.0.0.1:8000/employees> kemudian klik pada tautan **Detail** pada bagian tabel **Aksi**. Jika berhasil akan muncul tampilan berikut.



Detail Pegawai

Nama Lengkap	Muarifin
Email	muarifin@pens.ac.id
Nomor Telepon	085815667668
Tanggal Lahir	1988-12-03
Alamat	Taman Sukodono Regency Cluster 1
Tanggal Masuk	2025-09-21
Status	aktif

Update Data

1. Update data kedalam **tabel** pada **database** yang harus dilakukan adalah dengan memasukkan kode pada method **edit** (menampilkan **form** edit) dan **update** (menambahkan kode **update** data ke tabel).
2. Buka file **EmployeeController.php** dan tambahkan kode berikut pada method **edit**

```
public function edit(string $id)
{
    $employee = Employee::find($id);
    return view('employees.edit',compact('employee'));
}
```

3. Pada file **EmployeeController.php** dan tambahkan kode berikut pada method **update**

```
public function update(Request $request, string $id)
{
    $request->validate([
        'nama_lengkap' => 'required|string|max:255',
        'email'         => 'required|email|max:255',
        'nomor_telepon'  => 'required|string|max:20',
        'tanggal_lahir' => 'required|date',
        'alamat'         => 'required|string|max:255',
        'tanggal_masuk'  => 'required|date',
        'status'         => 'required|string|max:50',
    ]);
    $employee = Employee::findOrFail($id);
    $employee->update($request->only([
        'nama_lengkap',
        'email',
        'nomor_telepon',
        'tanggal_lahir',
        'alamat',
        'tanggal_masuk',
        'status',
    ]));
    return redirect()->route('employees.index');
}
```

4. Buat file `edit.blade.php` dalam direktori `employees` untuk membuat **Form Edit**. Tambahkan Kode berikut :

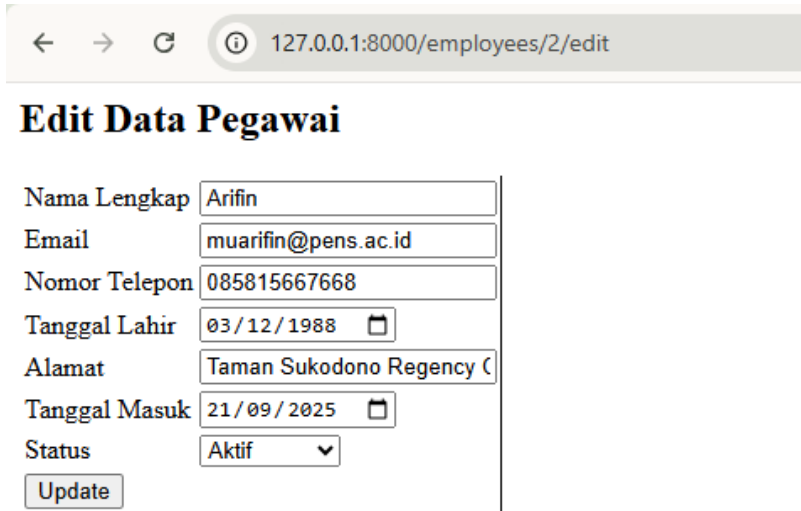
```
<h2>Edit Data Pegawai</h2>
<form action="{{ route('employees.update', $employee->id) }}" method="POST">
    @csrf
    @method('PUT')
    <table>
        <tr>
            <td>Nama Lengkap</td>
            <td><input type="text" name="nama_lengkap" value="{{ old('nama_lengkap', $employee->nama_lengkap) }}"></td>
        </tr>
        <tr>
            <td>Email</td>
            <td><input type="email" name="email" value="{{ old('email', $employee->email) }}"></td>
        </tr>
        <tr>
            <td>Nomor Telepon</td>
            <td><input type="text" name="nomor_telepon" value="{{ old('nomor_telepon', $employee->nomor_telepon) }}"></td>
        </tr>
        <tr>
            <td>Tanggal Lahir</td>
            <td><input type="date" name="tanggal_lahir" value="{{ old('tanggal_lahir', $employee->tanggal_lahir) }}"></td>
        </tr>
        <tr>
            <td>Alamat</td>
```

```

        <td><input type="text" name="alamat" value="{ old('alamat', $employee->alamat) }" ></td>
    </tr>
    <tr>
        <td>Tanggal Masuk</td>
        <td><input type="date" name="tanggal_masuk" value="{ old('tanggal_masuk', $employee->tanggal_masuk) }" ></td>
    </tr>
    <tr>
        <td>Status</td>
        <td>
            <select name="status">
                <option value="aktif" {{ old('status', $employee->status) == 'aktif' ? 'selected' : '' }}>Aktif</option>
                <option value="tidak aktif" {{ old('status', $employee->status) == 'tidak aktif' ? 'selected' : '' }}>Tidak
Aktif</option>
            </select>
        </td>
    </tr>
    <tr>
        <td colspan="2">
            <button type="submit">Update</button>
        </td>
    </tr>
</table>
</form>

```

5. Ujicoba dengan masuk ke halaman menampilkan data pada route <http://127.0.0.1:8000/employees> kemudian klik pada tautan **Edit** pada bagian tabel Aksi. Jika berhasil akan muncul **Form** untuk mengubah data berikut.



Edit Data Pegawai

Nama Lengkap

Email

Nomor Telepon

Tanggal Lahir

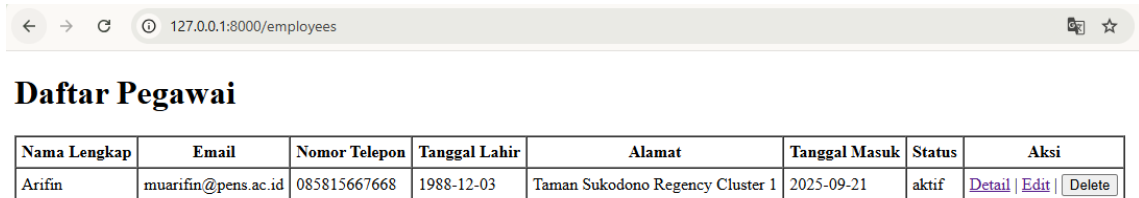
Alamat

Tanggal Masuk

Status

Pada contoh di atas adalah merubah nama yang sebelumnya *Muarifin* menjadi *Arifin* kemudian **Klik Update**.

6. Jika berhasil nama yang semula *Muarifin* akan menjadi *Arifin* pada tampilan data.



Daftar Pegawai

Nama Lengkap	Email	Nomor Telepon	Tanggal Lahir	Alamat	Tanggal Masuk	Status	Aksi
Arifin	muarifin@pens.ac.id	085815667668	1988-12-03	Taman Sukodono Regency Cluster 1	2025-09-21	aktif	Detail Edit Delete

Hapus Data

1. Proses hapus data menggunakan method `destroy`
2. Buka file `EmployeeController.php` dan tambahkan kode berikut pada method `destroy`.

```
public function destroy(string $id)
{
    $employee = Employee::find($id);
    $employee->delete();
    return redirect()->route('employees.index');
}
```

3. Ujicoba dengan masuk ke halaman menampilkan data pada route `http://127.0.0.1:8000/employees` kemudian klik pada button **Delete** pada bagian tabel **Aksi**. Jika berhasil akan di redirect ke halaman utama dan data telah terhapus



Daftar Pegawai

Nama Lengkap	Email	Nomor Telepon	Tanggal Lahir	Alamat	Tanggal Masuk	Status	Aksi
--------------	-------	---------------	---------------	--------	---------------	--------	------

Jika dalam implementasi nyata pertimbangkan anda menggunakan **soft delete**

Tugas ke 3

1. Kerjakan Percobaan di atas
2. Lakukan commit pada lokal repository anda pada *branch* `dev` dengan perintah `git add .` selanjutnya `git commit -m "CRUD tabel employee"`
3. Push ke remote repository anda. Pastikan sudah di branch `dev` lalu jalankan perintah `push origin dev`

Menyiapkan Template

Dari contoh **CRUD** tabel `employee` akan dicoba membuat tampilan untuk memisahkan **layout**, sehingga kedepan akan lebih mudah dalam mengelola tampilan

1. Buat file `master.blade.php` sebagai **master layout**. Layout ini berfungsi untuk tampilan semua layout yang nanti akan sama pada semua halaman akan dibuat pada layout **master** ini misalkan menu navigasi. Strukturnya adalah sebagai berikut.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>@yield('title', 'App Pegawai')</title>
</head>
<body>
    <header>
        <h1>@yield('page-title', 'App Pegawai')</h1>
        <nav>
            <ul>
                <li><a href="{{ url('/employee') }}">Employee</a></li>
                <li><a href="{{ url('/department') }}">Department</a></li>
                <li><a href="{{ url('/attendance') }}">Attendance</a></li>
                <li><a href="{{ url('/report') }}">Report</a></li>
                <li><a href="{{ url('/settings') }}">Settings</a></li>
            </ul>
        </nav>
    </header>
    <main>
        @yield('content')
    </main>
    <footer>
        <p>&copy; {{ date('Y') }} App Pegawai</p>
    </footer>
</body>
</html>
```

2. Sesuaikan tampilan pada halaman utama `employee` yaitu halaman untuk menampilkan data dengan penyesuaian halaman master berikut.

```
@extends('master')
@section('title', 'Daftar Pegawai')
@section('content')
<div class="container mt-5">
  <h1 class="mb-4">Daftar Pegawai</h1>
  <table border="1" cellpadding="5" cellspacing="0">
    <thead>
      <tr>
        <th>Nama Lengkap</th>
        <th>Email</th>
        <th>Nomor Telepon</th>
        <th>Tanggal Lahir</th>
        <th>Alamat</th>
        <th>Tanggal Masuk</th>
        <th>Status</th>
        <th>Aksi</th>
      </tr>
    </thead>
    <tbody>
      @foreach($employees as $employee)
        <tr>
          <td>{{ $employee->nama_lengkap }}</td>
          <td>{{ $employee->email }}</td>
```

```

        <td>{{ $employee->nomor_telepon }}</td>
        <td>{{ $employee->tanggal_lahir }}</td>
        <td>{{ $employee->alamat }}</td>
        <td>{{ $employee->tanggal_masuk }}</td>
        <td>{{ $employee->status }}</td>
        <td>
            <a href="{{ route('employees.show', $employee->id) }}">Detail</a> |
            <a href="{{ route('employees.edit', $employee->id) }}">Edit</a> |
            <form action="{{ route('employees.destroy', $employee->id) }}" method="POST" style="display:inline;">
                @csrf
                @method('DELETE')
                <button type="submit" onclick="return confirm('Yakin ingin menghapus?')">Delete</button>
            </form>
        </td>
    </tr>
    @endforeach
</tbody>
</table>
</div>
@endsection

```

3. Uji coba dengan membuka halaman utama menampilkan data `employees` jika berhasil maka tampilan akan seperti berikut.



App Pegawai

- [Employee](#)
- [Department](#)
- [Attendance](#)
- [Report](#)
- [Settings](#)

Daftar Pegawai

Nama Lengkap	Email	Nomor Telepon	Tanggal Lahir	Alamat	Tanggal Masuk	Status	Aksi
Arifin	muarifin@pens.com	085815667668	1999-12-03	Surabaya	2023-09-21	aktif	Detail Edit Delete

© 2025 App Pegawai

Menu **Navigasi** dalam bentuk vertikal karena hanya menggunakan HTML untuk memudahkan pembelajaran. Selanjutnya menu **navigasi** dapat digunakan pada fungsi tabel lainnya. Point penting pada tahap ini adalah memahami bagaimana alur **Blade Template** bisa menghubungkan antara halaman layout utama (master) dan layout konten.

Tugas ke 4

1. Kerjakan Percobaan di atas
2. Lakukan commit pada lokal repository anda pada *branch* `dev` dengan perintah `git add .` selanjutnya `git commit -m "Penambahan Master Layout"`
3. Push ke remote repository anda. Pastikan sudah di *branch* `dev` lalu jalankan perintah `push origin dev`

Menyiapkan Tabel

Pada tahap sebelumnya hanya menggunakan tabel `employees` sebagai percobaan. Penggunaan tabel `employees` dilakukan modifikasi dengan menghilangkan **foreign key**. Pada tahap ini akan dilakukan membuat semua tabel dengan menggunakan **migration**.

1. Perhatikan kembali desain tabel pada sub-bab **Desain Tabel**
2. Buat **migration** untuk membuat table `departments` dan `positions` karena sebelum membuat tabel `employees` kedua tabel tersebut harus dibuat terlebih dahulu. Dalam pembuatan **migration** masing-masing tabel, sekaligus membuat **Model** dari masing-masing tabel.

```
C:\Windows\system32\cmd.exe
D:\laragon\www\app-pegawai>php artisan make:model Department -m

INFO Model [D:\laragon\www\app-pegawai\app\Models\Department.php] created successfully.

INFO Migration [D:\laragon\www\app-pegawai\database\migrations/2025_09_21_151712_create_departments_table.php] created successfully.

D:\laragon\www\app-pegawai>php artisan make:model Position -m

INFO Model [D:\laragon\www\app-pegawai\app\Models\Position.php] created successfully.

INFO Migration [D:\laragon\www\app-pegawai\database\migrations/2025_09_21_151739_create_positions_table.php] created successfully.
```

3. Isikan `departments` migration dalam `method up` seperti berikut.

```
public function up(): void
{
    Schema::create('departments', function (Blueprint $table) {
        $table->id();
        $table->string('nama_departemen', 100);
        $table->timestamps();
    });
}
```

4. Isikan `positions` migration dalam `method up` seperti berikut.

```
public function up(): void
{
    Schema::create('positions', function (Blueprint $table) {
        $table->id();
        $table->string('nama_jabatan', 100);
        $table->decimal('gaji_pokok', 10, 2);
        $table->timestamps();
    });
}
```

5. Update tabel `employees` dengan menambahkan foreign key dari tabel `departments` dan `positions`.

```
C:\Windows\system32\cmd.exe
INFO Migration [D:\laragon\www\app-pegawai\database\migrations\2025_09_22_035007_add_
foreign_keys_to_employees_table.php] created successfully.

D:\laragon\www\app-pegawai>_
```

6. Isikan **migration** yang telah di generate pada **point 5** kode berikut.

```
public function up(): void
{
    Schema::table('employees', function (Blueprint $table) {
        $table->unsignedBigInteger('departemen_id')->after('tanggal_masuk');
        $table->unsignedBigInteger('jabatan_id')->after('departemen_id');

        $table->foreign('departemen_id')
            ->references('id')
            ->on('departemens')
            ->onDelete('cascade')

        $table->foreign('jabatan_id')
            ->references('id')
            ->on('positions')
            ->onDelete('cascade');
    });
}

public function down(): void
{
    Schema::table('employees', function (Blueprint $table) {
        $table->dropForeign(['departemen_id']);
        $table->dropForeign(['jabatan_id']);
        $table->dropColumn(['departemen_id', 'jabatan_id']);
    });
}
```

```
public function up(): void
{
```

```

Schema::table('employees', function (Blueprint $table) {
    $table->unsignedBigInteger('departemen_id')->after('tanggal_masuk');
    $table->unsignedBigInteger('jabatan_id')->after('departemen_id');

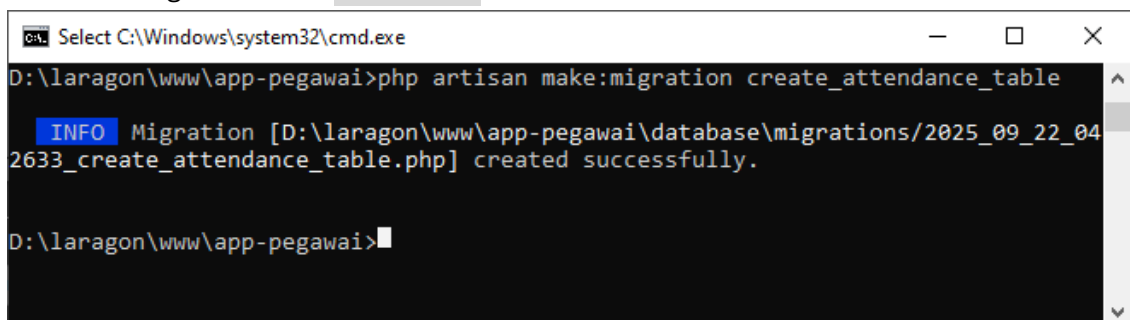
    $table->foreign('departemen_id')
        ->references('id')
        ->on('departments')
        ->onDelete('cascade');

    $table->foreign('jabatan_id')
        ->references('id')
        ->on('positions')
        ->onDelete('cascade');
});
}

public function down(): void
{
    Schema::table('employees', function (Blueprint $table) {
        $table->dropForeign(['departemen_id']);
        $table->dropForeign(['jabatan_id']);
        $table->dropColumn(['departemen_id', 'jabatan_id']);
    });
}

```

7. Membuat migration **tabel attendance**



```

C:\> Select C:\Windows\system32\cmd.exe
D:\laragon\www\app-pegawai>php artisan make:migration create_attendance_table

[INFO] Migration [D:\laragon\www\app-pegawai\database\Migrations\2025_09_22_042633_create_attendance_table.php] created successfully.

D:\laragon\www\app-pegawai>

```

```

public function up(): void
{
    Schema::create('attendance', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('karyawan_id');
    });
}

```



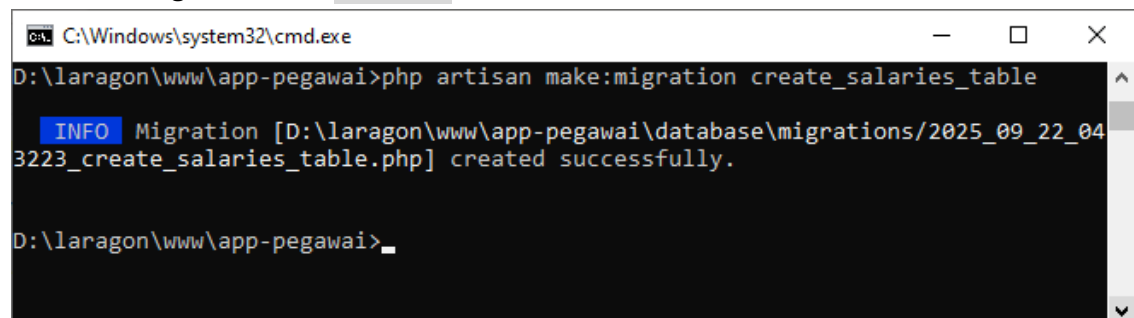
```

$table->date('tanggal');
$table->time('waktu_masuk')->nullable();
$table->time('waktu_keluar')->nullable();
$table->enum('status_absensi', ['hadir', 'izin', 'sakit', 'alpha']);
$table->timestamps();

// Foreign key constraint
$table->foreign('karyawan_id')
    ->references('id')
    ->on('employees')
    ->onDelete('cascade');
});
}

```

8. Membuat migration tabel **salaries**



```

C:\Windows\system32\cmd.exe
D:\laragon\www\app-pegawai>php artisan make:migration create_salaries_table

INFO Migration [D:\laragon\www\app-pegawai\database\migrations\2025_09_22_043223_create_salaries_table.php] created successfully.

D:\laragon\www\app-pegawai>

```

```

public function up(): void
{
    Schema::create('salaries', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('karyawan_id');
        $table->string('bulan', 10);
        $table->decimal('gaji_pokok', 10, 2);
        $table->decimal('tunjangan', 10, 2)->default(0);
        $table->decimal('potongan', 10, 2)->default(0);
        $table->decimal('total_gaji', 10, 2);
        $table->timestamps();

        // Relasi ke tabel employees
        $table->foreign('karyawan_id')
            ->references('id')
            ->on('employees')
    });
}

```

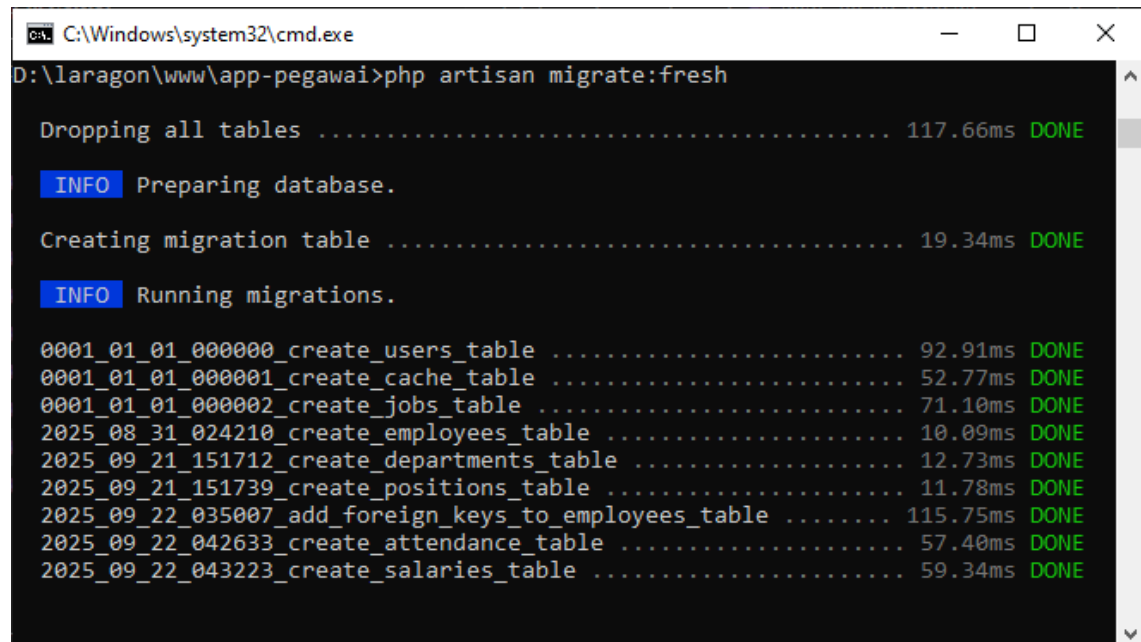
```

->onDelete('cascade');

});
}

```

9. Jalankan `php artisan migrate` jika ada eror bisa dijalankan `php artisan migrate:fresh` tetapi resikonya adalah menjalankan semua migrasi dari awal dan data dalam tabel akan hilang



```

C:\Windows\system32\cmd.exe
D:\laragon\www\app-pegawai>php artisan migrate:fresh

Dropping all tables ..... 117.66ms DONE

INFO Preparing database.

Creating migration table ..... 19.34ms DONE

INFO Running migrations.

0001_01_01_000000_create_users_table ..... 92.91ms DONE
0001_01_01_000001_create_cache_table ..... 52.77ms DONE
0001_01_01_000002_create_jobs_table ..... 71.10ms DONE
2025_08_31_024210_create_employees_table ..... 10.09ms DONE
2025_09_21_151712_create_departments_table ..... 12.73ms DONE
2025_09_21_151739_create_positions_table ..... 11.78ms DONE
2025_09_22_035007_add_foreign_keys_to_employees_table ..... 115.75ms DONE
2025_09_22_042633_create_attendance_table ..... 57.40ms DONE
2025_09_22_043223_create_salaries_table ..... 59.34ms DONE

```

10. Pada tahap ini semua tabel yang diperlukan telah dibuat